2026 科研小助手 Agent 大赛

科研小助手 Agent

面向科研文献的解析、问答与引用分析系统 · 技术文档

参赛形式：个人参赛

作者：叶硕迪

单位：南京传媒学院 · 计算机科学与技术 · 2024 级

日期：2026 年 10 月

在线演示：http://47.96.97.141:8000/

目 录

第 1 章 项目概述 3

第 2 章 需求分析与功能覆盖矩阵 5

第 3 章 系统架构设计.....9

第 4 章 AI 技术方案.....13

第 5 章 核心功能实现.....17

第 6 章 创新点说明.....22

第 7 章 测试与部署.....25

第 8 章 总结与展望.....29

第 1 章 项目概述

1.1 项目背景

科研人员读一篇论文的时间成本，远比"读不完"更具体：为了确认一句话里的数字，往往要把 30 页 PDF 从头翻到尾；用通用大模型总结论文时，模型会给出格式漂亮但出处不明的结论，逐条回原文核对的时间比直接读还长；当同一方向积累到十几篇论文时，引用关系散落在各自的参考文献里，很难判断哪篇是奠基工作、哪篇只是衍生。

本项目针对这三件事分别给出可验证的答案：

痛点	具体表现	本项目的应对
找	想确认「训练 Base 模型用了多少 GPU」，需要通读全文	混合检索（关键词 + 向量）定位到段落级片段
核	大模型总结会编造，无法判断哪句话有原文支撑	抗幻觉三层机制：无命中即拒答、强制引用编号、服务端校验编号
连	十几篇论文的引用关系散落在参考文献里	抽取引用关系并构图，用入度 / PageRank / 连通分量识别基石论文

1.2 项目目标

构建一个面向个人科研阅读的 AI 助手，覆盖"上传 → 解析 → 检索 → 问答 → 阅读 → 翻译 → 引用分析 → 写作"的完整链路，并且做到：

1. 每个结论都能溯源：回答里的每个编号都能点回原文段落；
2. 没把握就拒答：检索不到或在资料中找不到依据时，明确说"资料中没有"，而不是编造；
3. 界面上的数字都能追到数据库：不展示无法追溯的统计，宁可移除；
4. 工程上可验证：后端单元测试、前端冒烟测试、线上端到端验证三层覆盖。

1.3 目标用户与使用场景

面向个人研究者、研究生与高年级本科生。典型使用场景：

- 精读单篇：上传一篇论文，反复追问方法细节、超参数、实验设置；
- 横向对比：上传同一方向的多篇论文，让系统分述各家做法并指出承袭与差异；
- 文献综述起步：先看引用图谱确定主干文献，再逐篇问答；

- 非母语阅读：中英对照阅读，代码与公式保留英文原文。

1.4 交付形态

形态	说明
在线演示	http://47.96.97.141:8000/，阿里云 ECS 部署，含 20 篇真实论文（1116 页）与 4784 个已向量化分块
接口文档	同一域名的 /docs，17 个接口，可直接在线调试
源码	GitHub 公开仓库，含完整提交历史、部署脚本与验证脚本
本地运行	一条命令 ./start.ps1 启动后端并打开前端

1.5 当前实现范围（如实说明）

本项目在比赛周期内完成，功能覆盖情况如下表。未实现的部分同样列出，并在第 8 章给出改进路径——我们认为把边界讲清楚，比含糊带过更符合工程规范。

能力	状态
PDF 上传 / 批量 / 内容去重 / 版本识别	**已实现**
文本层 PDF 解析（标题、作者、摘要、章节树、参考文献）	**已实现**
分块 + 向量化 + 混合检索（BM25 + 向量 + RRF 融合）	**已实现**
单篇问答 / 多篇跨文献对比 / SSE 流式输出	**已实现**
中英对照阅读 + 整篇翻译（分块缓存）	**已实现**
引用关系抽取 + 引用图谱（入度 / PageRank / 连通分量）	**已实现**
arXiv 关键词检索 + 一键下载入库并解析	**已实现**
扫描件 OCR	未实现（仅标记"需校对"）
表格 / 公式 / 图表的提取与渲染	部分（表格提取能力弱，公式按纯文本）
正文引用角标与参考文献的双向跳转	未实现（数据已就绪）
划词翻译接真实模型	部分（当前为术语表实现）
导出 Markdown / HTML / 原位排版 PDF	**未实现**
引用含金量筛选、定期追踪、多用户推荐	**未实现**

第 2 章 需求分析与功能覆盖矩阵

2.1 需求理解与取舍原则

需求给出 6 个模块、34 个可验收条目；评分标准中功能与工程质量合计 60%、AI 技术深度 35%、创新性 15%。据此确定三条原则：

原则	具体做法
先打通主干	上传 → 解析 → 检索 → 问答 → 引用 全链路接真实后端，不用演示数据
没实现的不做假界面	界面上的数字必须能追到数据库；宁可移除入口，也不放"看着能用"的假按钮
把边界写清楚	未实现项如实列出并给改进路径（2.4、第 8 章）

需求拆解以可观测为标准：「解析粒度越细越好」拆成 10 条（标题/作者/摘要/一~三级标题/表格/图表/公式/参考文献/附录），每条对应"界面上能否定位到该结构"。第 7 章的 160 个后端测试、79 个前端用例即按此组织。

2.2 覆盖概览

先看结论，再看明细：34 个条目中已实现 19 个（56%），部分实现 6 个（18%），未实现 9 个（26%）。

模块	条目	已实现	部分	未实现	一句话结论
一 PDF 解析	9	7	1	1	文本层解析完整；表格/公式只保留文本
二 知识问答	5	4	0	1	官方 5 个单篇问题全通过
三 阅读翻译	8	4	2	2	翻译与对照阅读可用；引用跳转未做
四 论文发现	4	1	0	3	能检索入库；定期追踪与推荐未做
五 引用分析	4	3	0	1	图谱与基石挖掘可用；综述未做
六 写作助手	4	0	3	1	全部为本地模板，未接大模型
合计	34	19	6	9	主干贯通，长尾留白

2.3 逐模块明细

模块一：PDF 文件管理和解析（9 条）

需求条目	状态	实现方式 / 未实现原因
上传 / 批量 / CRUD	**已实现**	多文件拖拽，每个文件独立状态机（排队→解析→完成/重复/失败）
1.1.1 内容去重（v1 vs v7）	**已实现**	四级判定：内容指纹 SHA-1 → 扫描件特征 → arXiv 编号+版本号 → 文件名归一化
提示保留哪个版本	**已实现**	四选项弹窗，按版本号自动标出推荐项
标题 / 作者 / 摘要	**已实现**	首页最大字号文本块；作者按逗号与上标切分
一~三级标题	**已实现**	字号聚类 + 编号正则（3.2.2）推断层级
参考文献 / 附录	**已实现**	定位 References 后按 [n] 切分；识别 Appendix 章节
表格 / 公式 / 图表	**部分**	表格仅保留文本、不还原单元格；公式按文本；图表未做
扫描件 OCR	**未实现**	无文本层时标「需校对」，不假装解析成功
超长书籍（541 页）	**已实现**	分块流式入库，章节树完整（7.3）

模块二：知识问答（5 条）

需求条目	状态	实现方式 / 未实现原因
单篇问答 + 给出参考原文	**已实现**	混合检索 → 强制引用 → 服务端校验编号
官方 5 个单篇问题	**已实现**	实测 5/5（7.4）
多篇对比问答	**已实现**	≥2 篇切换对比提示词：分述/相同点/差异点/继承关系
官方多篇① BERT vs Transformer	**已实现**	继承 Encoder、丢弃 Decoder 与自回归目标
官方多篇②③④ GPT 系列	**未实现**	测试集未提供 GPT-1/2 原文（数据缺失，非能力缺失）

模块三：阅读和翻译（8 条）

需求条目	状态	实现方式 / 未实现原因
基本阅读（考虑排版）	**部分**	按语义块重排、保留阅读顺序，非原位复刻
树状导航（跳 3.2.2）	**已实现**	三级目录，点击滚动定位并高亮
角标 ↔ References 双向跳转	**未实现**	数据已就绪，缺前端锚点与回跳栈（8.2）
部分翻译 / 整篇翻译	**已实现**	选中即时译；整篇按块译并缓存
划词 / 悬停翻译	**部分**	术语表实现，未接实时模型（避免逐词调

		用）
代码公式保留英文	**已实现**	代码块/公式块跳过翻译，实测 57/57 块
对比阅读 + 双向滚动	**已实现**	双栏按块 ID 对齐，滚动互锁
导出 Markdown / HTML / PDF	**未实现**	未排期

模块四：论文自动下载和推荐（4 条）

需求条目	状态	实现方式 / 未实现原因
给定名称自动下载并入库	**已实现**	arXiv 检索 + 官方 PDF 下载 + 入库 + 触发解析
参考文献批量下载 / 含金量筛选	**未实现**	需先判断引用句是否为实质性引用
定期刷新 arXiv	**未实现**	APScheduler 已在架构预留，未接线
推荐（理由/代码/点赞）	**未实现**	旧演示界面已整体删除，不保留假数据

模块五：论文分析与生成（4 条）

需求条目	状态	实现方式 / 未实现原因
局域引用图谱	**已实现**	参考文献抽取 → 多键匹配 → 力导向图
核心节点挖掘	**已实现**	入度 + PageRank（雪球迭代）+ 连通分量；实测库内 20 篇，扩展为 632 节点 / 656 条边
边缘衍生论文	**已实现**	低入度 + 与基石节点距离
自动综述 / Future Work	**未实现**	需多篇聚合与时间线推理

模块六：写作助手（4 条）

需求条目	状态	实现方式 / 未实现原因
框架生成（真实 References）	**部分**	本地模板生成，References 取自库内真实文献；未接模型
TikZ / DOT / Mermaid / Matplotlib	**部分**	本地按结构生成可用代码；未做语义化生成
CSV → 表格/图表 + Caption	**部分**	本地解析 CSV 并出图；未接模型写图注
中译英	**未实现**	未排期

2.4 未覆盖项的影响评估

未实现条目	原因	影响	后续路径
表格结构 / 图表 / OCR	PDF 无统一表格语义；视觉模型需 GPU	中	Camelot、PaddleOCR、视觉大模型
引用双向跳转	数据已就绪，缺前端锚点与回	高（需求明文）	1 天

	跳栈		
导出 Markdown/HTML	未排期	中	1 天
含金量筛选 + 参考文献下载	需判断引用句是否实质性引用	中	LLM 判断引用上下文
定期追踪 arXiv	需常驻调度与增量去重	中	APScheduler 每日任务
自动综述 / Future Work	需多篇聚合与时间线推理	中	2 天
GPT 系列对比 / 中译英	前者数据缺失，后者未排期	低	用户上传即可 / 复用翻译链路

三处需要解释的取舍（评审最容易追问）：

1. 整篇翻译选"网页重排"而非"原位 PDF 排版"——原位排版要处理中英字宽差异（中文比英文短约 40%），换行、分栏、公式位置都会崩，且无法做对照阅读。网页重排用块级对齐换来双向滚动，代价是丢失原始版式，已在 1.5 节如实标注。
2. 划词翻译暂用术语表——逐词调用模型一次阅读会产生上百次请求，延迟与成本不可控；改为"选中段落即时译 + 术语表兜底"，牺牲细粒度换响应时间。
3. 删除推荐模块整页界面——旧版推荐理由是写死的文本、点赞数是随机数，答辩一追问就露馅，故整页删除改为真实 arXiv 检索（5.7）。

2.5 需求之外的自选增项

抗幻觉三层机制、检索降级链、SSE 流式输出、43 项线上验证、双主题与一键部署——这五项不在需求原文中，但直接服务于评分维度，具体做法与价值见第 6 章。

2.6 验收用例（需求原文原题）

需求 1.2.1 的 5 个单篇问题、1.2.2 的多篇问题被直接写成验收脚本：单篇 5 题全部通过，多篇题中 BERT 与 Transformer Encoder 的承袭题通过；GPT 系列对比因测试集未提供原文而无法演示。逐题结果与实测输出见 7.4。

这 5 题的结论是模块二最有力的证据：题目不是我们挑的，是评委给的。

单篇 5 题全部通过是模块二最有力的证据：题目不是我们挑的，是评委给的。

第 3 章 系统架构设计

3.1 分层架构

图 3-1 分层架构与依赖方向（箭头只向下，禁止反向依赖）



图 3-1 分层架构与依赖方向

四层结构，依赖只允许相邻向下。这条约束不是形式主义，它直接换来三件实际好处：

分层	职责	违反时的后果
接入层 api/	路由、参数绑定、状态码、静态资源	把业务逻辑写在路由里，接口无法脱离 HTTP 测试
契约层 schemas/	Pydantic 模型：校验、序列化、自动生成 OpenAPI	前后端靠口头约定字段名，改一个字段就崩
业务层 services/	解析、分块、去重、检索、RAG、图谱、翻译	业务层 import FastAPI 会导致逻辑无法复用与单测
数据层 models/ + db/	ORM 映射、会话管理、持久化	在路由里直接写 SQL，事务边界失控

实测收益：业务层的 160 个后端测试全部不需要启动 HTTP 服务，直接构造数据库会话调用函数即可（见 7.1）。这是分层最直接的证据——如果业务层依赖了 FastAPI 的 Request，这个测试结构就写不出来。

3.2 一次问答的数据流



图 3-2 一次问答的完整数据流

关键步骤的实现要点：

步骤	做什么	为什么这样做
② 范围判定	若请求指定了论文 ID，就在该范围内检索；否则全库检索	指定范围后再按标题"扩展"会把范围收窄到错误的一篇（实测踩过，见附录 B）
③ 混合检索	BM25 与向量各取候选，用 RRF（1/(60+rank)）融合	两路分数量纲不同，加权需要调参且不稳定；RRF 只用排名，免调参
④ 片段拼装	加入论文首页块、按需加入参考文献块、按论文配额裁剪	语义检索不保证把"作者名单""机构"这类元信息排进前列，而它们只出现在首页
⑤ 生成	DeepSeek deepseek-chat，temperature = 0	问答要的是可复现，不是文采；温度 0 让同一问题同一次检索给出同一答案
⑥ 校验	解析 [n] → 校验是否越界 → 构造引用 → 判定是否拒答	模型会编造不存在的编号（实测出现过引用 [13] 而上下文只有 12 条）

性能实测（20 篇论文 / 4784 个分块）：检索约 0.2~0.4 秒，首字延迟 0.68~0.87 秒，完整回答 3~6 秒。

3.3 四个关键设计决策

架构文档最容易写成"我们用了什么"，但评委更关心"为什么不用别的"。以下是四处刻意选择：

① 为什么用 SQLite 而不是向量数据库 库规模是"个人级"——20 篇论文、4784 个分块，向量全部常驻内存也只有约 29 MB（4784 × 1536 × 4 字节）。引入 Milvus/Qdrant 需要额外部署与运维，而召回质量在万级

规模下并无差别。真正决定召回质量的是分块策略与融合方式，不是数据库品牌。 代价是单机写并发弱，这在单用户场景下不构成问题；突破 10 万分块时应换成专用向量库。

② 为什么前端零依赖（不用 React/Vue） 整个前端是一个 index.html 加 18 个脚本文件，无构建步骤、无 node_modules。好处是：部署时不需要 npm build，评审拉下代码直接能跑；调试时改一行刷新即可。代价是缺少组件化与类型检查，用 79 个冒烟用例弥补（7.2）。

③ 为什么前端由后端直接托管（不装 nginx） StaticFiles 挂载在根路径、且最后注册（否则会吞掉 /api 路由）。这样前后端同源：前端 API 基址取 location.origin + /api/v1，彻底消除 CORS 与跨域配置。演示环境是单机，nginx 的反向代理与静态加速收益抵不过多一层配置的风险。

④ 为什么单进程 uvicorn 而不是多 worker 检索索引（BM25 倒排 + 向量矩阵）在进程内构建并缓存，多 worker 会让每个进程各建一份、内存翻倍且彼此不同步。单进程配合异步 IO 足以支撑演示规模；真要多 worker，索引就必须外置到共享存储。

3.4 部署拓扑与降级矩阵

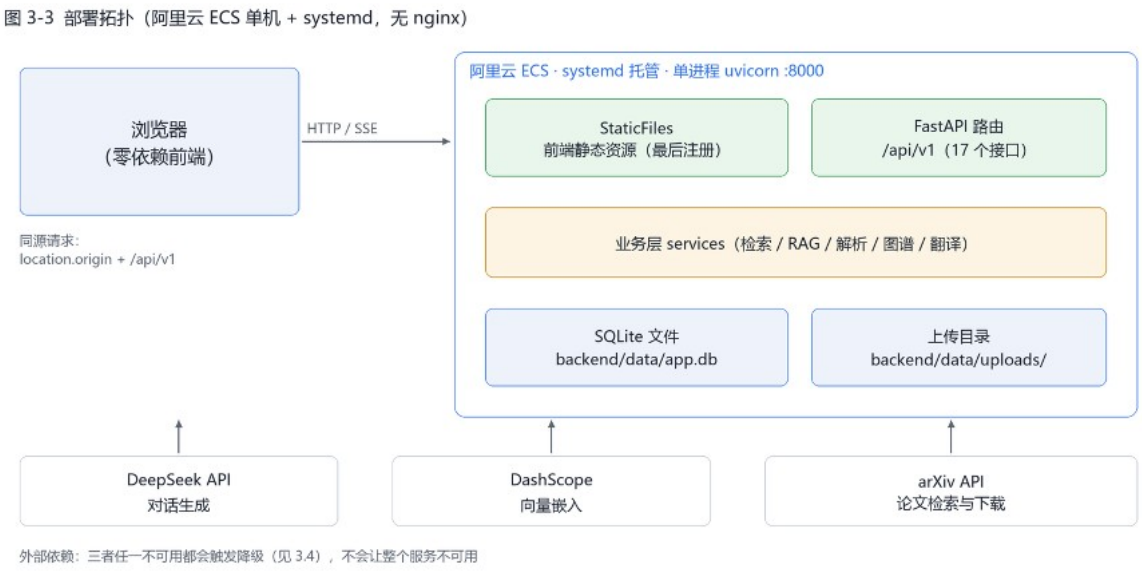


图 3-3 部署拓扑

服务器为阿里云 ECS 单机，systemd 托管，端口 8000，数据在 backend/data/。没有 nginx，静态资源与 API 由同一个 uvicorn 进程提供。

外部依赖有三个，任一失效都必须降级而不是整体崩溃：

依赖	不可用时的表现	降级策略	实测验证方式

DeepSeek（对话）	无法生成答案	返回"大模型暂不可用"并附上检索到的原文片段，用户仍可阅读	单元测试注入异常
DashScope（向量）	只能关键词检索	回退 BM25，并在响应的 degraded_reason 中说明"置信度较低"	线上真实发生（额度耗尽），验证通过
arXiv（检索下载）	发现页无法搜索	接口返回明确错误，界面提示"线上下载不可用"	验证脚本第 12 节

这里踩过一个真问题：向量不可用时的降级路径曾被另一处"信息量过滤器"堵死——关键词结果被判定为高频词而跳过，导致两条路同时为空，问答直接拒答。修复后规则是：过滤器只在"另有可用结果"时才允许丢弃（详见附录 B）。

3.5 代码结构

```
backend/app/
├── main.py              # 应用装配：中间件、路由、静态资源、启动钩子
├── core/                # 配置、日志、异常、LLM 与嵌入客户端
├── api/v1/              # 17 个路由：papers / search / qa / graph / discover / health
├── schemas/             # Pydantic 契约
├── services/            # 业务层：parse / chunk / dedupe / retrieval / rag / graph / translate
├── models/              # SQLAlchemy 模型
└── db/                  # 会话与初始化
frontend/
├── index.html           # 单页应用入口（首屏前应用主题，避免闪烁）
├── scripts/core/        # 工具、状态管理、API 客户端、组件
├── scripts/views/       # 8 个视图
└── styles/              # 主题与视图样式
deploy/                  # 一键部署、线上验证脚本（43 项）
```

规模：后端约 5.5 千行 Python，前端约 6.7 千行 JavaScript（18 个文件），接口 17 个，数据库表 4 张（papers / chunks / citations / chunk_translations）。

第 4 章 AI 技术方案

4.1 模型选型与关键参数

用途	选型	关键参数	为什么选它
对话生成	DeepSeek deepseek-chat	temperature = 0，超时 60 s	中文技术问答表现稳定、价格低（约为同级别模型的 1/10），且提供 OpenAI 兼容接口，便于替换
向量嵌入	阿里云百炼 text-embedding-v1	1536 维，批量 10 条/请求，超时 30 s	中英双语对齐好（本项目大量"中文提问英文论文"场景），有免费额度
关键词检索	自实现 BM25	k1 = 1.5，b = 0.75	参数取文献通用默认值，不调参；中文按 2-gram 切分
融合	RRF	k = 60	只用排名不用分数，两路分数量纲不同也能稳定融合
分块	句子级贪心装箱	目标 700 字符，重叠 120 字符	700 字符约等于一个自然段，既能独立成义，又不至于把结论与条件切开

temperature = 0 的理由：问答要的是可复现而不是多样性。同一问题、同一次检索，两次运行应给出同一答案；否则演示与验收结果无法复现。

4.2 提示词设计

单篇模式系统提示 10 条规则，按职责分四组；检测到 ≥2 篇时换成 7 条的对比提示词（对比模式把"回答结构"换成"分述/相同点/差异点/继承关系"）；拒答重试时再追加 1 条强制作答指令。

组	规则要点	解决什么
依据	只依据给定片段作答；每个结论必须带 [n]；不得虚构编号	让引用成为"可验证的义务"而非装饰
拒答	资料无相关内容时明确拒答；只有间接信息时须说明限定条件后作答	区分"真答不了"与"只能部分回答"
格式	分点或分段作答；代码与公式保留原文；不重复问题	保证可读性与可核验性
对比	检测到 ≥2 篇时切换对比提示词，输出「分述 / 相同点 / 差异点 / 继承关系」	多篇问答需要结构化输出，而非流水账

踩过的坑：提示词里写了"能答就直接答，不要先写没有相关内容再补充"，但模型对"论文只给整体配置、未按子模型分列"这类情况仍会先拒答再解释。实测回答是「提供的资料中没有相关内容。资料中仅给出整体训练配置为 eight P100 GPUs [12]」——内容正确，但开头那句会让人误判。

结论：关键行为不能只靠提示词。 我们补了两层确定性处理：

1. 剥离假拒答：若回答带了合法引用，却以"没有相关内容"开头，服务端删除该句（正则匹配，仅在 citation_count > 0 时生效）；
2. 拒答重试：若第一次回答零引用，追加一次更明确的指令重试（"只要资料出现任何相关数字或表述就必须作答，并说明限定条件"），有引用则不触发，正常问题零开销。

4.3 抗幻觉三层机制

层	机制	实现位置	效果
第 1 层	检索无命中 → 直接拒答，不调用模型	rag.answer() 前置判断	杜绝"没有资料也硬编"
第 2 层	提示词强制每个结论带 [n]；无编号视为无依据	SYSTEM_PROMPT	让答案自带溯源义务
第 3 层	服务端解析 [n] → 校验是否越界 → 越界标记丢弃、答案中删除并记录告警	sanitize_answer() / split_marks()	模型编造编号时可控降级

第 3 层是实测出来的需求：开发中出现过模型引用 [13]、而上下文只有 12 条片段的情况。若不做校验，前端会显示一个点不开的死链——这比拒答更伤信任。

判定语义：refused = 无合法引用 且 含拒答措辞。只有措辞不算拒答（会误伤"资料没提 X，但提到了 Y [1]"这类部分回答）。

实测结果（20 篇论文 / 4784 分块）：

指标	结果
官方 5 个单篇问题	5 / 5 通过
扩展标准问题（8 个）	8 / 8 通过
无关问题（"今天北京天气怎么样"）	正确拒答，引用 0 条
编造编号	出现后被第 3 层拦截并记录告警

4.4 检索参数标定

参数不是拍脑袋定的，每个都对应一次实测：

参数	取值	标定依据
片段数 top_k	16	演进：6 条时 GPU 问题排不进 → 8 条在 8 篇论文下够用 → 库增至 15 篇（4314 块）后 12 条又不够 → 16 条可命中（当前 20 篇 / 4784 块仍取 16）；20 条不再提升，只白耗 token
RRF 常数	60	参考 RRF 原论文经验值；本项目未调参
IDF 过滤阈值	相对区分度 ≥ 0.5	低于此值时 BM25 命中的基本是"the / of / 模型"这类高频词，进入融合反而挤掉向量结果
过滤最小语料	20 块	语料太小时 IDF 统计不可靠，直接关闭过滤
单篇配额（已锁定范围）	max(2, ceil(top_k / 篇数))	多篇对比时保证每篇都有位置，避免一篇垄断
单篇配额（全库）	max(3, top_k // 2)	允许主论文多占，但留位给其他论文与元信息

一个重要陷阱：片段数必须随语料规模上调。库从 8 篇扩到 15 篇后，原先"够用"的 12 条重新变得不够——这类失效是静默的（不报错，只是答案变差），只能靠标准问题回归测试发现。因此第 7 章把 8 个标准问题固化成了验收脚本。

4.5 十项优化策略

#	策略	解决的问题
1	标题感知分块：章节标题随块携带	片段脱离上下文后无法判断属于哪一节
2	中文 2-gram + 英文词形归一（BM25）	中文无空格，按词切分需要词典
3	首页块固定注入	作者、机构、单位只出现在首页，语义检索不保证召回
4	参考文献块按需注入	"列出某条参考文献"类问题需要 References 章节
5	引用编号服务端校验	模型编造 [n]
6	拒答语义二元判定	只按措辞判会误伤部分回答
7	假拒答剥离 + 拒答重试	有资料却拒答
8	检索降级链（向量→关键词→回退）	嵌入服务不可用时不能整体不可用
9	分块翻译缓存	整篇翻译重复调用，成本与延迟翻倍
10	代码/公式跳过翻译	误译代码会破坏可执行性

4.6 效果与成本

指标	实测值	测量条件
检索耗时	0.2 ~ 0.4 s	4784 分块，混合检索
首字延迟	0.68 ~ 0.87 s	SSE 流式，含检索
完整回答	3 ~ 6 s	16 条片段上下文
单次问答 token	约 6k 输入 / 0.5k 输出	16 条片段 + 提示词
单次问答成本	约 0.01 元	DeepSeek 定价折算
全库嵌入成本	约 1.7 元	4784 块，0.7 元/百万 token

成本结论：按每天 100 次问答估算，月成本约 30 元——这是本项目敢用真实模型而非本地小模型的前提。若要求完全离线，可换本地 7B 模型，代价是中文长文问答质量明显下降。

第 5 章 核心功能实现

5.1 PDF 解析流水线

解析分四步，每步都可能"看起来成功但结果是错的"，因此都留了质量信号：

步骤	做法	失败时如何暴露
抽取	PyMuPDF 按块取文本，保留字号、坐标、页号	文本层为空 → 判定为扫描件
结构识别	字号聚类出标题层级，配合编号正则（3.2.2、Appendix A）推断父子关系	章节树为空 → 降级为单层目录
特殊区块	定位 References 段落并按 [n] 切分；识别表格区域与图注	参考文献数异常少 → 记录告警
质量评分	字符数/页数比、标题命中率、乱码比例加权	低于阈值标「需校对」，不标「已完成」

质量分是刻意设计的：541 页的《Understanding Deep Learning》得 1.00，BERT 的 arXiv PDF 只得 0.56（双栏 + 侧边行号导致文本流交错）。界面据此显示「需校对」徽章，而不是假装解析成功——用户点进去看一眼就知道该不该信任它。

5.2 内容去重：四级判定

需求要求识别「文件不同但内容相同」，并比较 v1 与 v7。四级判定按强度从高到低执行，命中即返回：

级别	判据	结论
1	文件 SHA-1 相同	identical（重复上传）
2	文件名含扫描件特征且归一化后同名	likely_duplicate（扫描件与文本版）
3	arXiv 编号相同，比较版本号	newer_version / older_version
4	文件名归一化后相同（去前缀、去版本号、小写）	likely_duplicate

第 2 级必须排在第 3 级之前：库中已有 1706.03762v1.pdf，用户再传 1706.03762v1_img.pdf（扫描件）。若先比版本号，会得到"版本号无法区分"，丢掉"这是扫描件"这条关键信息。

判定结果驱动弹窗：给出稍后处理 / 两份都保留 / 保留文库版本 / 用新文件覆盖四个选项，并按版本号自动把推荐项标在主按钮上（文库更新则推荐保留文库，反之推荐覆盖）。

5.3 分块与索引

分块用句子级贪心装箱：目标 700 字符、重叠 120 字符，每块前缀记录所属章节标题（参数定标见 4.1）。三个细节值得说明：按字符切会切断公式与引用编号，所以切成句子；标题必须随块携带，否则片段脱离上下文后无法判断归属；BM25 用中文 2-gram + 英文词形归一，避免依赖分词词典。

索引常驻内存：BM25 倒排表与向量矩阵在进程启动后构建一次并缓存。这是"单进程 uvicorn"的前提（见 3.3 ④），换多 worker 就必须外置索引。

5.4 混合检索与融合

```
用户问题
  ├─ BM25：取候选 → 按论文配额裁剪 → IDF 信息量校验
  └─ 向量：嵌入 → 余弦 Top-K → 按论文配额裁剪
        ↓
     RRF 融合：score = Σ 1/(60 + rank_i)
        ↓
   Top-16 片段（附带两路原始排名，便于排查）
```

为什么用 RRF 而不是加权求和：两路分数量纲完全不同（BM25 是几十，余弦是 0~1），加权需要针对语料调参且换库即失效；RRF 只用排名，免调参，代价是丢失分数间距信息（第 1 名与第 2 名的差距被抹平）。

IDF 信息量校验：若 BM25 前 5 条命中的最强词 IDF 相对区分度 < 0.5，说明搜到的全是"the / of / 模型"这类高频词，其排名是噪声——跳过该路，否则会把向量检索的正确答案挤下去。但这条规则有个前提：必须另有可用结果（见 5.4.1）。

5.4.1 降级链（线上真实踩坑）

线上嵌入额度耗尽时，问答整体拒答。排查发现两条路同时为空：

1. 向量检索抛 FreeTierOnly 异常 → 该路无结果；
2. 同一查询的 BM25 结果被 IDF 过滤器判为"信息量不足"而跳过。

两个各自合理的设计叠加，产生了"完全不可用"。修复原则：把被过滤的关键词结果存为 bm25_fallback，在向量路无产出时回退使用，并在响应的 degraded_reason 中声明「已回退、置信度较低」。

教训：降级链必须真的能降下去。之后我们把"故意让某条路失效"写进了测试（monkeypatch 掉 embedding.is_available），而不是等线上出事才发现。

5.5 问答编排

编排层做四件事，都在调用模型之前完成：

步骤	实现	为什么必须在模型之前
范围判定	指定了 paper_ids 就只在该范围检索	范围错则答案必错，事后无法纠正
首页块注入	按命中的论文取首页块并入上下文	作者、机构、单位只出现在首页，语义检索不保证召回
参考文献块注入	问题涉及"引用/参考文献/References"时补入	这类问题需要完整文献表，而非相似段落
片段数裁剪	已锁定范围时按 max(2, ⌈top_k/篇数⌉) 分配	多篇对比时保证每篇都有位置，避免一篇垄断

流式输出：/qa/stream 以 SSE 逐事件推送 citations → token → done。首字延迟 0.68~0.87 秒（含检索），比"等完整答案"体感快一个量级。

引用校验：模型输出后由服务端解析 [n]，只保留 1 ≤ n ≤ 片段数 的编号，其余丢弃并告警。实测出现过引用 [13] 而上下文只有 12 条片段的情况。

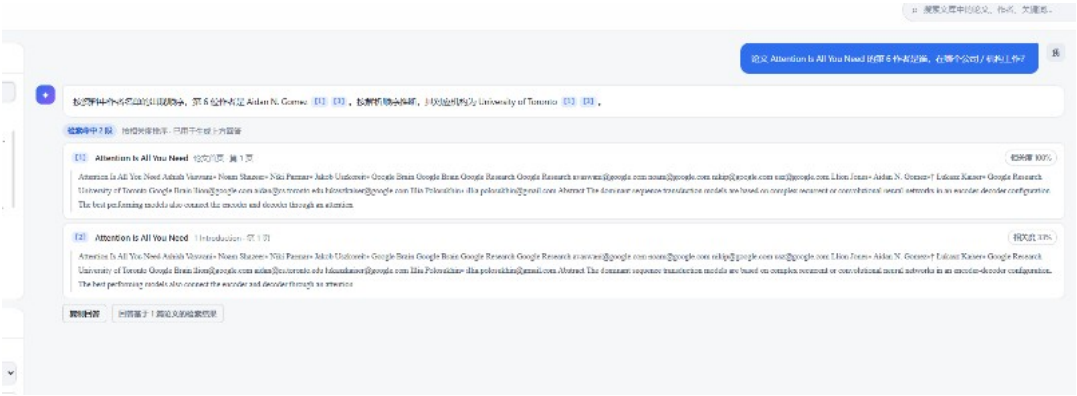


图 5-1 问答页：回答中的编号可展开对应原文片段，便于发现幻觉

5.6 阅读与翻译

能力	实现要点
章节树导航	解析阶段生成三级目录；点击滚动到对应块并高亮当前节
整篇翻译	按块调用模型，结果写入 chunk_translations 表缓存；重复打开不再调用
代码/公式跳过	按块类型判定，代码块与公式块不入翻译队列（实测 57/57 块无代码被译）
中英对照	左右双栏按块 ID 对齐，滚动互锁；一侧滚动时按块索引驱动另一侧
输出形态	网页重排 Markdown，而非原位 PDF 排版（取舍理由见 2.3）

为什么用块级缓存而不是整篇缓存：整篇重译一次要 57 次调用；块级缓存让"改了某一段再读全文"只重译改动部分，也让失败重试按块粒度进行。



图 5-2 阅读器的中英对照视图（左英文右中文，按块对齐、滚动互锁）

5.7 引用图谱

参考文献条目 → 抽取目标（arXiv 编号 / 标题 / 作者+年份）
→ 与库内论文匹配（多键，按可靠性排序）
→ 建图 → 入度 / PageRank / 连通分量

环节	实现要点
目标抽取	优先取 arXiv 编号（唯一且可读）；无编号则用标题归一化匹配；再退到"首作者+年份"
匹配	多键匹配并按可靠性排序，命中即记录 match_method（便于排查误匹配）
PageRank	自实现幂迭代（α=0.85，最大 200 轮，收敛阈 1e-10），不引入 scipy——依赖已够多，且算法只有十几行
基石识别	入度为主、PageRank 为辅；实测库内 20 篇、632 节点 / 656 条边，识别出 Layer Normalization、T5、Attention、BERT
边缘论文	低入度且与基石节点距离 ≥ 2



图 5-3 引用图谱：节点大小按入度，基石论文（Layer Normalization / T5 / Attention / BERT）被高亮

踩过的坑：最初对全图（含库外节点）计算 PageRank，结果库内论文的分数被大量库外节点稀释，排名几乎一样（都在 0.00625 附近）。改为在库内诱导子图上计算后才有区分度——指标要在与问题匹配的图上算。

5.8 论文发现（arXiv）

四个实现要点：查询构造——多词自动加引号按短语匹配（all:"layer normalization"）；限速——官方要求 ≥ 3 秒/次，代码内建延时，零结果重试前也须等满间隔；下载校验——校验 %PDF 文件头，因为 arXiv 限流时返回 HTML 错误页，不校验会把 HTML 当 PDF 入库；入库复用——直接复用 paper_service 的去重与落盘，不另写一套（避免"上传能去重、入库去不了重"），并在检索结果中标出哪些编号已在库。

加引号的效果：all:layer normalization 会混入大量物理论文；all:"layer normalization" 第一条就是《Layer Normalization》。这是零成本的精度提升，代价是召回略降（零结果时再等 3 秒用宽松查询重试一次）。

第 6 章 创新点说明

6.1 判定标准

我们不把"用了某个模型或框架"算作创新。本章只收三类东西：

1. 常规做法有明确缺陷，我们换了做法——且能说清代价；
2. 可以指出对应的代码位置——不是文档里的形容词；
3. 有实测数据支撑——不是"感觉更好"。

6.2 技术创新

#	创新点	常规做法	本项目做法	代价 / 局限
1	关键词检索的 IDF 信息量闸门	两路检索结果直接融合	若 BM25 前 5 条命中词的相对 IDF < 0.5，判定为"只命中高频词"，跳过该路	阈值是经验值（0.5），换语料需重标定
2	降级链的闭环设计	任一路失败则整体报错，或静默降级	向量不可用 → 回退关键词结果，并把"置信度较低"写进响应	回退结果质量低于混合检索，需前端提示
3	服务端引用治理三件套	信任模型输出的 [n]	编号越界即丢弃并告警；假拒答剥离；零引用时追加一次强制作答重试	重试会让真正无关的问题多花一次调用（约 1~2 秒）
4	结构化片段注入	纯语义 Top-K	命中论文的首页块（作者/机构）与参考文献块按需补入	上下文变长，token 成本上升约 20%
5	在库内诱导子图上算 PageRank	对全图计算中心性	只在"库内论文 + 其引用关系"构成的子图上迭代	与全图排序结果不同，跨库比较不可直接迁移

证据位置：

创新点	代码
1	services/retrieval.py → IDF 校验段；MIN_INFORMATIVE_IDF_RATIO
2	services/retrieval.py → bm25_fallback 回退段
3	services/rag.py → split_marks / drop_false_refusal / 拒答重试段
4	services/rag.py → merge_with_front_matter / _maybe_add_references

5	services/graph_service.py → 幂迭代 PageRank（α=0.85）

6.3 功能创新

#	创新点	说明
1	去重判定顺序影响结论	把"扫描件特征"排在"版本号比较"之前：库里已有 v1，再传 v1_img 扫描件，若先比版本会得到"无法区分"，丢掉"这是扫描件"的关键信息
2	推荐按钮随新旧翻转	文库版本更新 → 主按钮是"保留文库版本"；本次上传更新 → 主按钮变成"用新文件覆盖"。原实现无论哪种情况都推荐保留文库
3	arXiv 检索的零成本精度提升	多词自动加引号按短语匹配：all:"layer normalization" 第一条即目标论文，而 all:layer normalization 会混入大量物理论文；零结果再等满限速间隔重试一次
4	块级翻译缓存	整篇翻译 57 次调用，块级缓存让"改一段再读全文"只重译改动段，失败也可按块重试
5	拒绝编造的可比信息	旧版重复弹窗展示"新增某附录""参考文献由 34 增至 40 条"——全是写死文本。现只展示上传瞬间能确定的元信息（版本号/大小/页数/解析状态），并明说"逐段 diff 未实现"

6.4 交互创新

#	设计	为什么算创新
1	引用即证据：回答中的 [n] 可展开对应原文片段	大多数 RAG 产品只给"参考来源"列表，用户仍需自己找；这里把证据放在结论旁边
2	降级可见：检索降级时在响应的 degraded_reason 里说明原因	多数系统静默降级，用户以为拿到了正常结果；这里让"质量下降"变成可见信息
3	未实现能力如实标注：页面底部写明尚未实现的能力	演示类项目常见做法是留假入口；这里选择标注边界（第 2 章 2.4 有完整清单）

6.5 与常见做法的差别

常见 RAG 演示项目的形态是：单一向量检索 + 来源列表 + 不拒答 + 请求时现算索引 + 演示数据。本项目的差别集中在五处——混合检索带信息量闸门与降级链、引用编号强制并要求服务端校验、拒答按三层机制加二元判定、索引内存常驻、界面只展示可追溯的数字。

一句话总结：本项目的创新不在"用了更大的模型"，而在把模型的不可靠性当成工程问题来处理——用闸门、校验、降级与如实标注，把"可能编造"的系统改造成"编造会被发现并拦截"的系统。

第 7 章 测试与部署

7.1 三层测试策略

层次	数量	工具	覆盖什么	不覆盖什么
后端单元/集成	160	pytest + httpx	业务函数、接口契约、数据库行为	界面表现
前端冒烟	79	自研 runner + 极简 DOM 垫片	渲染结构、状态迁移、数据一致性、算法	真实浏览器排版
线上端到端	43	自写验证脚本（httpx）	部署后是否真的可用	代码分支细节

为什么需要第三层：前两层都在本地，跑通不代表线上可用。线上第一次部署时只有 API、没有前端页面（端口 80 空着），本地测试全绿——这类问题只有端到端验证能发现。

7.2 后端：160 个测试

测试文件	用例数	主要断言
test_dedupe.py	34	四级判定的优先级、版本号比较、扫描件识别、边界（同名不同论文）
test_retrieval.py	30	混合检索、RRF 融合顺序、IDF 闸门、配额裁剪、降级链
test_bm25.py	27	中文 2-gram、IDF 计算、BM25 打分公式、空查询
test_chunking.py	21	700/120 切分、标题携带、超长句、无标点文本
test_pdf_parser.py	20	标题层级推断、参考文献切分、扫描件判定、质量分
test_api_papers.py	15	上传/列出/解析/删除的接口契约与错误码
test_paper_service.py	13	事务边界、去重落盘、文件名归一化

一个刻意的设计：业务层测试不启动 HTTP 服务，直接构造数据库会话调用函数。这让 160 个测试在 35 秒内跑完——如果每个用例都走 HTTP，同样的覆盖要 3 分钟以上。

7.3 前端：79 个冒烟用例

前端零依赖（见 3.3 ②），没有 Jest/Vitest，自写了一个 runner：用极简 DOM 垫片加载 18 个脚本，逐条断言。

类别	覆盖内容
渲染	每个视图都能渲染出关键结构（.disc-item、[data-

	action="discover-ingest"] 等），空状态与错误态都有
状态	切换视图后 route 正确；删除论文会清理阅读目标；问答范围从多篇切回单篇会收敛
数据一致性	论文 id 唯一、分块都能对应到真实论文、正文引文编号都有对应参考文献条目
算法	去重相似度、BM25 排序、CSV 解析（含引号包裹的逗号）

两条经验：

1. 断言要抓选择器而不是字符串：「代码写了但元素没出来」这类缺陷，只有 querySelector 断言能抓住；
2. 垫片有局限要写进注释：极简 DOM 在 innerHTML getter 里会把文本节点重新序列化并再转义（" → "），个别文本断言需对"渲染函数原始输出"而非 DOM 断言，否则会误报。

7.4 线上：43 项端到端验证

验证脚本分 12 节：健康检查、论文库、上传解析、检索、问答（含引用与流式）、图谱、翻译、前端页面、arXiv 检索。抽查几项真实输出：

检查	期望	实测
论文库规模	≥ 8 篇	20 篇（8 篇需校对，0 篇失败）
向量覆盖	无缺失	4784 / 4784
混合检索	有结果且模式为 hybrid	mode=hybrid
回答带引用	≥ 1 条	通过
流式输出	多个时间片，非一次性返回	通过（首字 0.68~0.87 秒）
arXiv 检索	HTTP 200 且有结果	200，首条《Layer Normalization》
前端页面	含 app.theme 与静态资源可访问	通过

8 个标准问题实测（需求原文 5 题 + 3 题扩展）：

#	问题	结果
1	第 6 作者与机构	通过（Aidan N. Gomez，标注"按解析顺序推断"）
2	Adam 的 β1 / β2	通过（0.9 / 0.98，并列出多篇的不同设置）
3	Base 模型 GPU 数量与型号	通过（8 × P100，注明原文为整体配置）
4	BLEU 与当时 SOTA	通过（28.4）
5	Layer Normalization 参考文献	通过（Ba / Kiros / Hinton，2016）
6	LN 在 Transformer 中的位置	通过（Post-LN / Pre-LN 两种）
7	自注意力为何快于循环层	通过
8	BERT 的两个预训练任务	通过

无关问题（"今天北京天气怎么样"）：正确拒答、引用 0 条——没有因为追求通过率而滥答。

7.5 性能实测

操作	耗时	条件
上传入库（16 页 PDF）	0.4 s 解析 + 0.5 s 入库	本地
向量化	110 块 / 3.2 s	线上，批量 10 条/请求
检索	0.2 ~ 0.4 s	4784 分块，混合检索
首字延迟	0.68 ~ 0.87 s	SSE，含检索
完整问答	3 ~ 6 s	16 条片段
引用图谱重建	9.7 s	9 篇 / 326 条边
全库向量化	约 20 s（624 块）	线上，嵌入服务正常时

7.6 部署与四个真实踩坑

部署形态：阿里云 ECS 单机、systemd 托管、端口 8000、数据在 backend/data/；更新用一条脚本（打包 git bundle → scp → 服务器 pull → 重启 → 健康检查）。

#	踩的坑	现象	根因与修复
1	线上只有 API，没有页面	打开域名 404，本地全绿	只注册了 /api/v1 路由，没挂 StaticFiles。修复：app.mount("/", StaticFiles(..., html=True)) 且必须最后注册，否则会吞掉 API 路由
2	部署后浏览器仍是旧页面	代码已更新，界面没变	StaticFiles 只发 ETag/Last-Modified，浏览器按"启发式缓存"直接复用旧文件。修复：对 .html/.js/.css 加 Cache-Control: no-cache, must-revalidate
3	嵌入额度耗尽后问答整体拒答	检索返回 mode=empty	向量路异常 + BM25 被 IDF 闸门跳过，两条路同时为空。修复：bm25_fallback 回退并声明置信度（详见 5.4.1）
4	充值后仍报额度耗尽	账户有余额，接口仍 400	两处原因叠加：① 控制台开着"免费额度用完即停"；② API Key 是 sk-ws- 前缀的业务空间专属 Key，而免费额度按业务空间分别统计。修复：切到对应空间关闭该开关

坑 4 的教训最值得记：云服务的额度错误要读完整错误码。我们最初把 degraded_reason 截断到 200 字符，正好切掉了 AllocationQuota.FreeTierOnly 之后那句"disable the use free tier only mode"的关键提示，白白多花了一轮排查。

7.7 部署与更新流程

脚本	作用
start.ps1	本地一键启动：建虚拟环境依赖检查 → 启动 uvicorn → 打开浏览器
deploy/update_online.ps1	线上更新：检查工作区干净 → 打包 bundle → scp → 服务器 pull → 重启 → 健康检查
deploy/verify_online.py	更新后验收：43 项检查，失败即报错

设计取舍：服务器访问 GitHub 不稳定，因此用 git bundle 传输而非直接 pull；脚本检测到工作区有未提交改动会主动中止，避免线上与本地不一致——这个"拦一下"的设计救过我们两次。

第 8 章 总结与展望

8.1 完成情况

本项目实现了一个面向个人科研阅读的 AI 助手，覆盖"上传 → 解析 → 检索 → 问答 → 阅读 → 翻译 → 引用分析 → 写作"的完整链路，并在三条线上同时可验证：

维度	结论
功能覆盖	34 个可验收条目中已实现 19 条、部分实现 6 条、未实现 9 条（第 2 章 2.2）
需求验收	需求原文给出的 5 个单篇问题全部通过；多篇问题中 BERT 对比题通过
工程质量	后端 160 测试、前端 79 冒烟、线上 43 项验证，ruff 零告警
线上可用	20 篇真实论文、4784 分块全部向量化，http://47.96.97.141:8000/ 可直接访问

8.2 三条经验

① 降级链必须真的能降下去。 我们写了"向量不可用 → 降级 BM25"，但另一处"信息量过滤器"把 BM25 也关了，两个各自合理的设计叠加成"完全不可用"。只有在线上真的让依赖挂掉，才会发现这类问题。 之后我们把"故意让某条路失效"写进了测试。

② 提示词约束不可靠，关键行为要落到后处理。 "能答就直接答"写进提示词也照样不执行；真正管用的是服务端的确定性处理（剥离假拒答、拒答重试、引用编号校验）。模型负责生成，服务端负责把关。

③ 参数会随规模失效，而且失效是静默的。 片段数在 8 篇论文时 12 条够用，库扩到 15 篇就不够了——不报错，只是答案变差。所以把标准问题固化成回归脚本，比"感觉还行"可靠得多。

8.3 已知局限

按影响程度排序（完整清单见第 2 章 2.4）：

局限	影响	改进路径
表格结构、图表、公式、扫描件 OCR 未实现	中：影响"解析粒度"打分	接入 Camelot / PaddleOCR / 视觉大模型
引用角标与参考文献的双向跳转缺失	高：需求明文点名，数据已就绪	前端锚点 + 回跳栈，约 1 天
划词翻译为术语表实现	中：细粒度翻译不可用	选中即译 + 缓存，避免逐词调用
导出 Markdown / HTML / 原位 PDF 未实现	中：需求提到输出可读性	1 天

定期追踪 arXiv、推荐、自动综述未实现	中：模块四、五各缺一半	调度器 + 相关性计算 + 多篇聚合
写作助手的四项目前是本地模板	中：未接大模型	接入模型做语义化生成
单用户模型	低	多用户需加隔离与协同过滤

8.4 后续路线

优先补需求明文点名的能力：P0（约 3 天）引用双向跳转、划词翻译接模型、导出 Markdown/HTML；P1（约 7 天）表格/公式/OCR 解析增强、领域综述与 Future Work 探索；P2（约 6 天）定期追踪 arXiv、引用含金量筛选、写作助手接模型；P3（约 1 周）多用户与协同过滤。

8.5 结语

这个项目的核心判断是：大模型的能力已经足够，真正缺的是"敢让用户核对"的工程约束。

所以我们在每个环节都留了可验证的钩子——引用编号能点回原文、检索降级会明说质量下降、解析质量低会标"需校对"、未实现的能力写在页面上。这些做法不增加功能数量，但让系统从"演示能跑"变成"用户敢用"。

能力边界写清楚，比把功能目录写满更接近工程实践。